

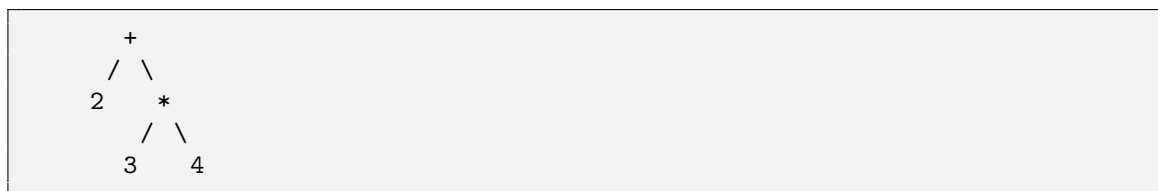
Bug Hunt v2-4: Expression Interpreter

What the program does

This problem consists of **two modules** plus a driver:

- **parser.erl** — a recursive-descent parser that turns a list of tokens into an **AST** (abstract syntax tree). An AST is a tree-shaped representation of the expression that captures its structure: which operator applies to which operands, in what order. The parser's job is to read the flat token list and produce a tree that the evaluator can walk recursively.

For example, the expression `2 + 3 * 4` becomes the tree:



In Erlang, this tree is represented as the nested tuple `{op, '+', {num, 2}, {op, '*', {num, 3}, {num, 4}}}`. The `*` is *deeper* in the tree than the `+`, which is what makes `*` bind tighter — the evaluator visits the inner `{op, '*', ...}` before combining its result with the `2` via the outer `+`.

The grammar the parser implements:

```

expr    := term (('+' | '-') term)*      -- left-associative
term    := unary (('*' | '/') unary)*    -- left-associative
unary   := '-' unary | primary          -- unary minus, right-
      associative
primary := number | '(' expr ')'
```

- **eval.erl** — evaluates an AST. Evaluation walks the tree recursively: to evaluate `{op, Op, L, R}`, you evaluate `L`, evaluate `R`, then combine the two results using `Op`. The evaluator handles literal integers, binary operations (`+`, `-`, `*`, `/` using integer division), and unary negation.
- **main.erl** — feeds 8 test expressions through the parser and evaluator. The expressions are given as pre-tokenized lists.

The three AST node shapes the parser produces and the evaluator consumes:

- `{num, N}` — an integer literal (leaf node)
- `{op, Op, L, R}` — a binary operation, where `Op` is one of `'+'`, `'-'`, `'*'`, `'/'`, and `L` and `R` are themselves AST nodes (the left and right subtrees)

Programming Language Fundamentals

Oregon State University Cascades

Spring 2026

- $\{\text{neg}, E\}$ — unary negation, where E is an AST node

The intended semantics:

- Arithmetic operators have standard precedence: $*$ and $/$ bind tighter than $+$ and $-$. All four operators are **left-associative**, so $10 - 3 - 2$ parses as $(10 - 3) - 2$, not $10 - (3 - 2)$.
- **Unary minus** binds tighter than any binary operator: $-2 * 3$ parses as $(-2) * 3$, not $-(2 * 3)$.
- Unary minus is right-associative for repeated application: $--7$ parses as $-(-7) = 7$.

The catch

This program contains **exactly 4 bugs** that prevent it from producing the expected output. The bugs are all *semantic* — the program compiles without errors and runs to completion without crashing.

This is the hardest problem in the v2 set. Three specific traps to watch for:

1. **Two of the bugs produce the same wrong answer on the most obviously failing test.** Fixing only one of them will leave that test still wrong (possibly with a *different* wrong answer than before). Do not assume your fix didn't work just because the test still fails — you may need to disambiguate with a different test that isolates each bug independently.
2. **One bug is masked by another on multiple tests.** Several tests that *look correct* in the as-shipped state will start showing wrong values after you fix the masking bug. Do not assume your fix introduced those regressions — the underlying bug was always there; it was just being canceled out.
3. **Some bugs are in `parser.erl` but their symptoms appear in eval-domain outputs (wrong values), and at least one bug is in `eval.erl` but cross-references with a parser bug.** Don't commit to one file as “the problem location” too early.

You may not run the program until you have submitted your answer. Trace by reading.

Expected output

```
2 + 3 * 4 = 14
(2 + 3) * 4 = 20
10 - 3 - 2 = 5
12 / 6 / 2 = 1
5 - 2 = 3
-5 = -5
5 - -3 = 8
-2 * 3 = -6
```

Notes on the expected output:

- $10 - 3 - 2 = 5$: subtraction is left-associative, so this is $(10 - 3) - 2 = 7 - 2 = 5$. If you computed $10 - (3 - 2) = 9$, re-read the spec.

Programming Language Fundamentals

Oregon State University Cascades

Spring 2026

- $12 / 6 / 2 = 1$: same logic. $(12 / 6) / 2 = 2 / 2 = 1$.
- $5 - 2 = 3$: simple binary subtraction. Use this as a diagnostic — if this is wrong but $10 - 3 - 2$ is *also* wrong, the cause may be elsewhere than where you first looked.
- $-5 = -5$: basic unary minus.
- $5 - -3 = 5 - (-3) = 5 + 3 = 8$: binary minus applied to a unary minus.
- $-2 * 3 = (-2) * 3 = -6$: unary minus binds tighter than $*$.

Your write-up

For each bug, record:

1. **Location** — file name and line number(s).
2. **Diagnosis** — in 1–2 sentences, what is wrong.
3. **Fix** — the corrected line(s).
4. **Symptom** — which line(s) of the expected output differ from what the buggy program actually prints, and why this specific bug causes that specific symptom. If two bugs produce the same symptom on a test, explain how you distinguished them. If a bug has no direct visible symptom because it is masked by another, explain how you knew it was there.